

## INFORMATION ZUR VORLESUNG

# REST – Distributed Parcel System

Modul 3IT-VSIT-50 · Verteilte Systeme und Internet der Dinge  
Thema Eine REST-API entwerfen und implementieren  
Dozent Dr. Ulrich Winkler  
Semester 5. Semester · Informationstechnik

## REST – Distributed Parcel System

Übung zur REST-Vorlesung (5. Semester, Informationstechnik, DHSN Dresden). Sie entwerfen die API eines kleinen Paketdienstes und implementieren sie in Python. Die Aufgabenstellung steht in `uebungen/01_parcel_service/aufgaben.md` (daneben liegt dieselbe Datei als PDF).

Der rote Faden ist nicht CRUD. Es geht um die Eigenschaften, die REST in einem verteilten System überhaupt erst nützlich machen: idempotente Methoden, bedingte Requests, Nebenläufigkeit, Caching und definiertes Verhalten, wenn das Netz unzuverlässig ist.

### Keine Datenbank – und das ist der Punkt

Ihre Daten liegen in **Dateien**, nicht in einem DBMS. Das ist eine bewusste Entscheidung: Mit einer relationalen Datenbank wären die interessanten Aufgaben dieser Übung in je einer Zeile SQL erledigt.

Mit Dateien müssen Sie sie selbst bauen. Sie werden dabei auf genau die Probleme stoßen, die ein Datenbanksystem intern löst:

| Was eine DB Ihnen schenkt          | Was Sie hier selbst brauchen                                    |
|------------------------------------|-----------------------------------------------------------------|
| <code>UNIQUE</code> -Constraint    | atomares Anlegen ( <code>O_CREAT</code>   <code>O_EXCL</code> ) |
| <code>SELECT ... FOR UPDATE</code> | Dateisperre ( <code>fcntl.flock</code> )                        |
| atomarer Commit                    | in Temp-Datei schreiben, dann <code>os.replace</code>           |
| Versionsspalte                     | ein <code>version</code> -Feld, das Sie selbst pflegen          |

Die Umgebungsvariable `DATA_DIR` zeigt auf das Datenverzeichnis ( `loesung/data` ). Lesen Sie den Pfad daraus, statt ihn in den Code zu schreiben. Das Verzeichnis existiert noch nicht – Ihr Code legt es an.

### Was der Container mitbringt – und was nicht

| Bestandteil                                               | wofür                                                  |
|-----------------------------------------------------------|--------------------------------------------------------|
| Python 3.12                                               | Laufzeit                                               |
| <code>uv</code>                                           | Paketmanager                                           |
| <code>curl</code> , <code>httpie</code> , <code>jq</code> | HTTP-Requests von Hand stellen                         |
| <code>uuidgen</code>                                      | Idempotency-Keys erzeugen                              |
| REST Client (VS-Code-Erweiterung)                         | <code>.http</code> -Dateien direkt im Editor ausführen |

| Bestandteil             | wofür                          |
|-------------------------|--------------------------------|
| Thunder Client, Postman | Klick-Oberflächen mit Historie |

Bewusst **nicht** enthalten sind das Python-Projekt und seine Abhängigkeiten. Es gibt keine `pyproject.toml`, keine `uv.lock`, keine `requirements.txt` und kein Web-Framework. Das legen Sie selbst an.

Framework und Bibliotheken sind Ihre Wahl; FastAPI ist ein bequemer Einstieg, Flask geht genauso. Für die Dateiablage brauchen Sie **keine** zusätzliche Bibliothek – `json`, `os`, `pathlib`, `fcntl` und `hashlib` aus der Standardbibliothek reichen vollständig aus.

## Uvicorn

Uvicorn ist ein ASGI-Webserver für Python. Er nimmt HTTP-Verbindungen entgegen und führt darüber Python-Webanwendungen aus, insbesondere Frameworks wie FastAPI und Starlette.

```
uv add fastapi "uvicorn[standard]"
```

ASGI steht für Asynchronous Server Gateway Interface. Es ist eine Schnittstelle/Standard zwischen einem Python-Webserver und einer Python-Webanwendung.

```
# Entwicklung – ein Prozess, lädt bei Änderungen neu
uv run uvicorn main:app --reload --host 0.0.0.0 --port 8000

# Für echte Nebenläufigkeit vier Prozesse
uv run uvicorn main:app --workers 4 --host 0.0.0.0 --port 8000
```

Nicht beides zusammen angeben: Uvicorn bricht dann nicht ab, sondern schreibt nur `WARNING: "workers" flag is ignored when reloading is enabled.` und läuft mit einem einzigen Prozess weiter.

## Wie Sie arbeiten

### GitHub Codespaces (empfohlen – nichts zu installieren)

1. **Code - Codespaces - Create codespace on main.**
2. Warten, bis der Container gebaut ist (beim ersten Mal einige Minuten).
3. Terminal öffnen und loslegen.

### Lokal in VS Code

Docker und die Erweiterung **Dev Containers** installieren, Repo klonen, Ordner öffnen und **Reopen in Container** wählen. Unter Windows brauchen Sie WSL2 mit Docker Desktop; ohne das nehmen Sie bitte Codespaces.

**Hinweis für Windows-Nutzer, die lokal arbeiten:** `fcntl` gibt es nur auf POSIX-Systemen. Im Dev-Container (Linux) ist das kein Problem – arbeiten Sie also im Container und nicht daneben.

## Aufbau

```
.devcontainer/          Container und Werkzeuge
uebungen/
├─ 01_parcel_service/
│  └─ aufgaben.md      Aufgabenstellung (auch als PDF)
```

```
├─ curl-beispiele.sh    curl in acht Beispielen
└─ requests.http        dieselben Anfragen für REST Client / Postman
loesung/                Ihr Projekt – legen Sie es selbst an
└─ data/                Ihre Dateiablage zur Laufzeit (nicht committen)
```